

MiniJava:

Sprache, Interpretierer
und Compiler

Heinz Dobler

Version 1, 2025



Java



MiniJava

Bildquelle: <https://iconduck.com/>

- Sprache MiniJava: Beispiel und Grammatik
- MiniJava-**Interpreter**: Architektur und Datenfluss
- Exkurs: HP-Taschenrechner
- MiniJava-**Compiler**: Architektur und Datenfluss (in zwei Varianten)
- Virtuelle MiniJava-**Maschine**: Befehlssatz und Architektur
- Disassembler (in zwei Varianten)
- Gegenüberstellung: Quelltext und Bytecode
- Attributierte Grammatik für MiniJava-**Interpreter**
- Attributierte Grammatik für MiniJava-**Compiler**

MiniJava- Beispielprogramm:

SVP.mj:

```
void main() {  
  
    int a, b, cs;  
  
    a = read();  
    b = read();  
    cs = a * a + b * b;  
    print(cs);  
}
```

MiniJava-Grammatik in Wirth'scher EBNF:

```
MiniJava = "void" "main" "(" ")" "{"  
           [ VarDecl ]  
           StatSeq  
           "}" .
```

```
VarDecl = "int" ident { "," ident } ";" .
```

```
StatSeq = Stat { Stat } .
```

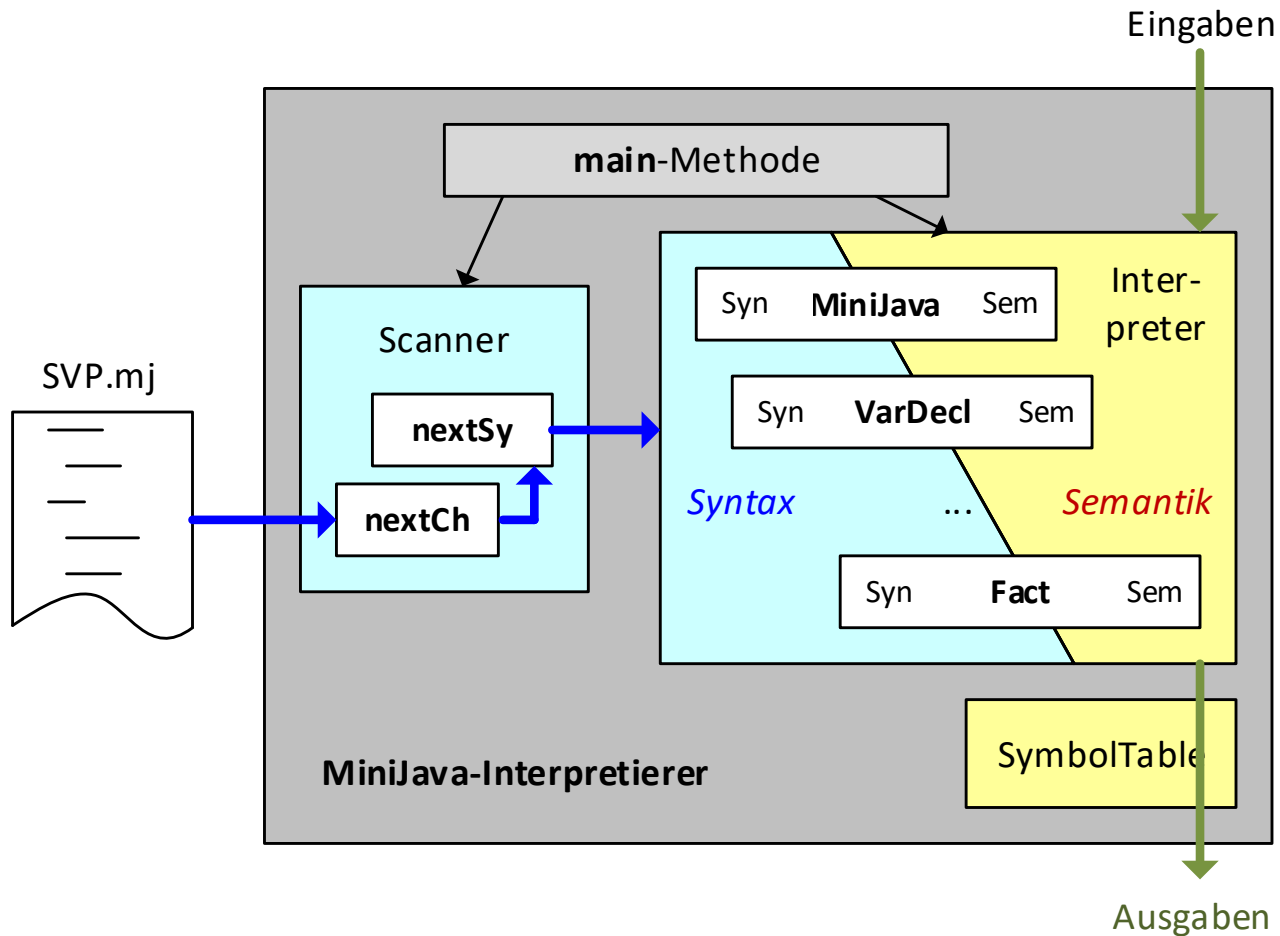
```
Stat = [ ident "=" Expr  
        | "print" "(" Expr ")"  
        ] ";" .
```

```
Expr = Term { "+" Term | "-" Term } .
```

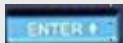
```
Term = Fact { "*" Fact | "/" Fact } .
```

```
Fact = number  
       | ident  
       | "read" "(" ")"  
       | "(" Expr ")" .
```

Architektur und Datenfluss des MiniJava-Interpreterers



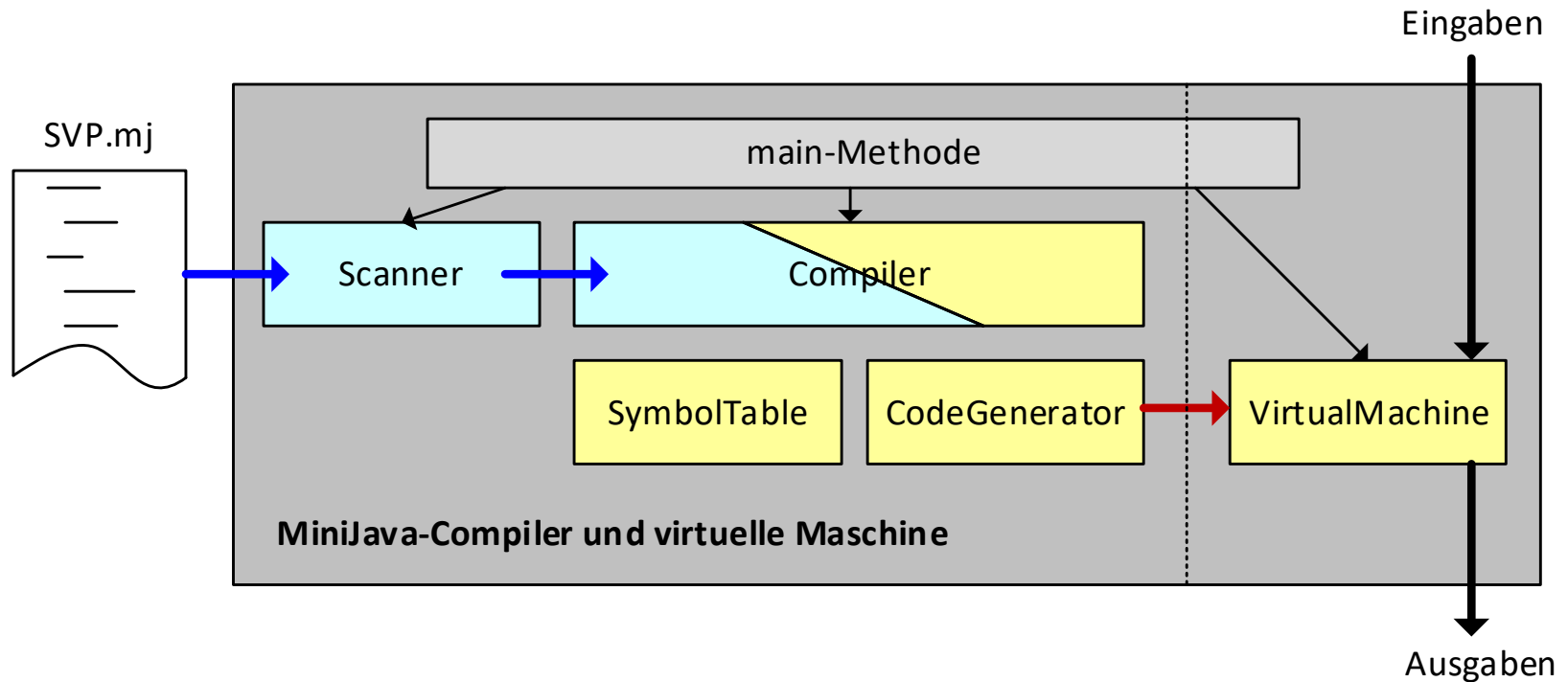
- Hewlett Packard (HP) baute lange Zeit (von 1968 bis 1986) ausgezeichnete Taschenrechner, beginnend mit HP-35, siehe z. B. <http://www.hpmuseum.org>
- Spezielle Eigenschaft: Kellerspeicher (*stack*) zur Auswertung von Ausdrücken, Tiefe auf vier Elemente beschränkt
- *Enter*-Taste für die *Push*-Operation
- Eingabe der Ausdrücke in *Umgekehrt polnischer Notation* (UPN), z. B. wird $17 + 4$ berechnet mittels

17  4 

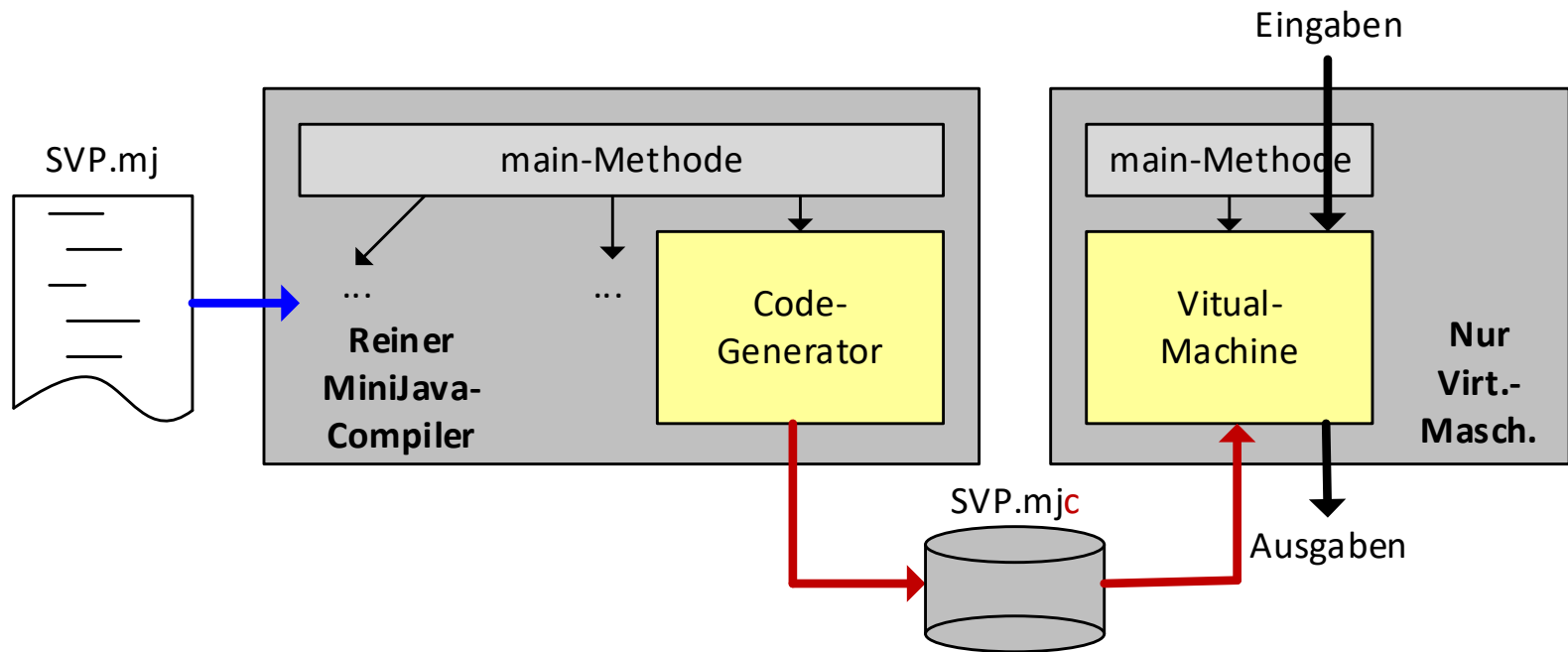


<http://www.hpmuseum.org/simulate/hp35sim/calc.html>

Architektur und Datenfluss des MiniJava-Compilers in Variante 1:
Compiler und virtuelle Maschine befinden sich in einem Programm

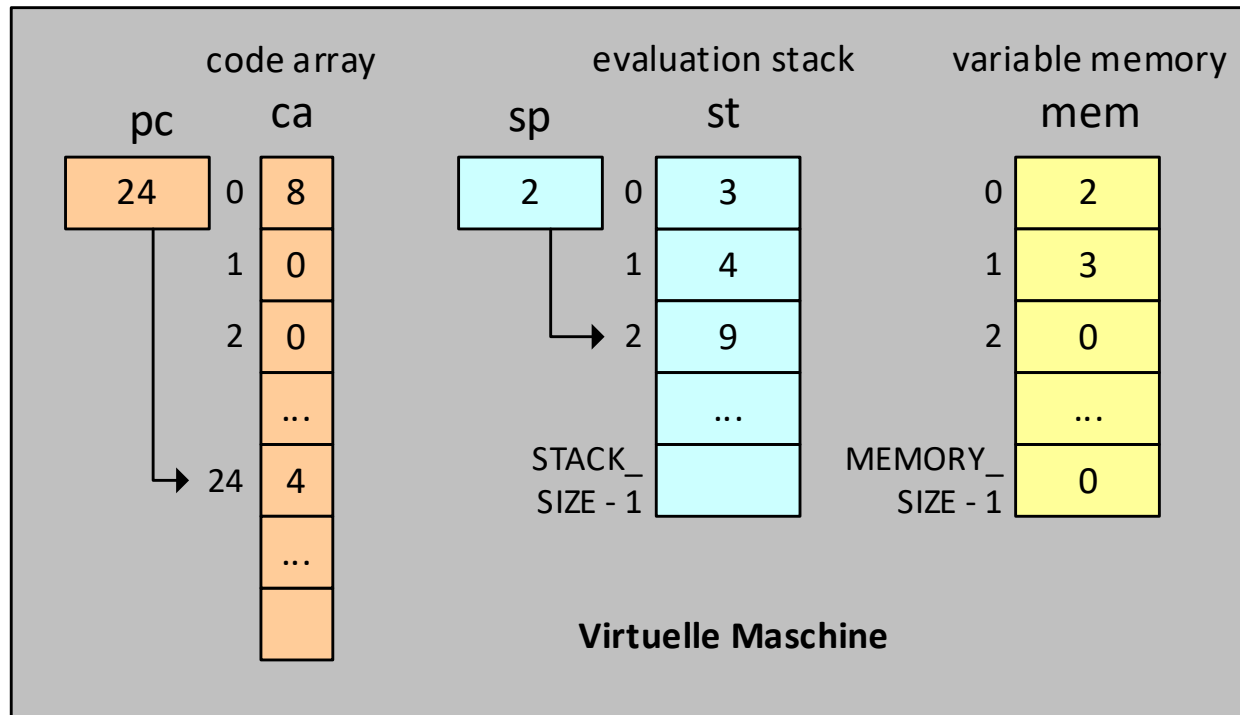


Architektur und Datenfluss des MiniJava-Compilers in Variante 2:
Compiler u. virtuelle Maschine in zwei separaten Programmen



Operationscode		Operand (4 Bytes)	Bedeutung
Wert	Mnemonic (1 Byte)		
0	LOAD_CONST	num	Lade die Zahl <i>num</i> auf den Stack
1	LOAD_VAL	addr	Lade die Zahl an der Adresse <i>addr</i> auf den Stack
2	LOAD_ADDR	addr	Lade die Adresse <i>addr</i> auf den Stack
3	STORE	-	Hole eine Zahl und eine Adresse vom Stack, speichere die Zahl an dieser Adresse ab
4	ADD	-	Hole zwei Zahlen vom Stack und lege deren <i>Summe</i> auf den Stack
5	SUB	-	Hole zwei Zahlen vom Stack und lege deren <i>Differenz</i> auf den Stack
6	MUL	-	Hole zwei Zahlen vom Stack und lege deren <i>Produkt</i> auf den Stack
7	DIV	-	Hole zwei Zahlen vom Stack und lege deren <i>Quotient</i> auf den Stack
8	READ	-	Lies eine Zahl von der Eingabe und lade diese auf den Stack
9	PRINT	-	Hole eine Zahl vom Stack und schreibe diese in die Ausgabe
10	END	-	Beende das Programm

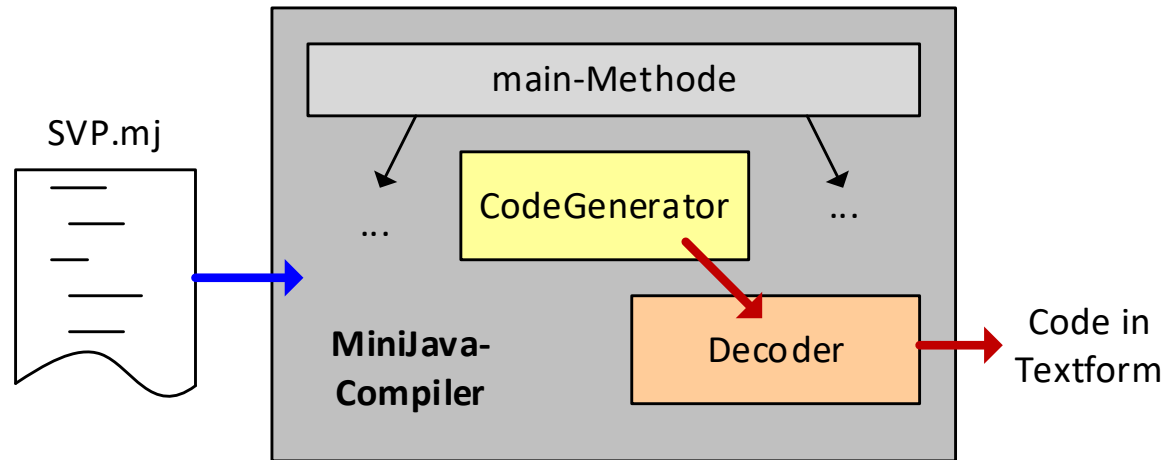
Architektur der virtuellen Maschine anhand eines Beispiel-Zustands



Decodierer ("Disassembler")

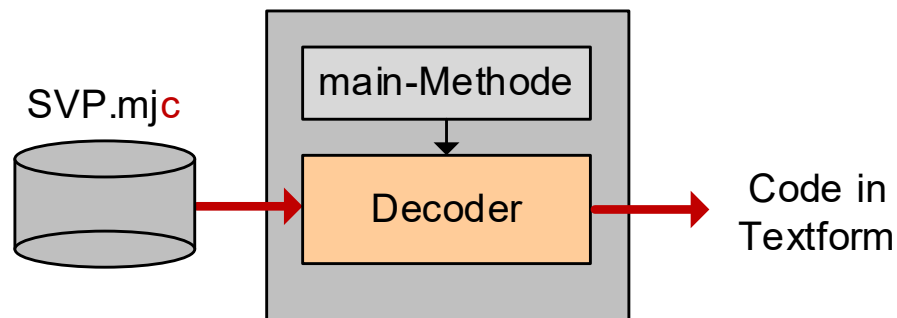
Variante 1:

**Decodierer
integriert in
den Compiler**



Variante 2:

**Decodierer als
eigenständiges
Programm**



Gegenüberstellung: Quelltext und Bytecode

Quelltext	Bytecode	
	Adr.	OpCode Operand
void main() { // 0 1 2 int a, b, cs; a = read();	0: LOAD_ADDR 5: READ 6: STORE	0
b = read();	7: LOAD_ADDR 12: READ 13: STORE	1
cs = a * a + b * b;	14: LOAD_ADDR 19: LOAD_VAL 24: LOAD_VAL 29: MUL 30: LOAD_VAL 35: LOAD_VAL 40: MUL 41: ADD 42: STORE	2 0 0 1 1
print(cs);	43: LOAD_VAL 48: PRINT	2
}	49: END	

Vergleich: MiniJava- mit Java- und .net-Bytecode

MiniJava 	Java 	MS .net 
MiniJava-Quelltext	Java-Quelltext	C#-Quelltext
<code>int x, a, b, c; c = b*(a/5+10);</code>	<code>int a, b, c; c = b*(a/5+10);</code>	<code>int x, a, b, c; c = b*(a/5+10);</code>
MJ-Bytecode	Java-Bytecode	CIL-Bytecode
<code>LOAD_ADDR 3</code>	–	–
<code>LOAD_VAL 2</code>	<code>iload_2</code>	<code>ldloc.2</code>
<code>LOAD_VAL 1</code>	<code>iload_1</code>	<code>ldloc.1</code>
<code>LOAD_CONST 5</code>	<code>iconst_5</code>	<code>ldc.i4.5</code>
<code>DIV</code>	<code>idiv</code>	<code>div</code>
<code>LOAD_CONST 10</code>	<code>bipush 10</code>	<code>ldc.i4.s 10</code>
<code>ADD</code>	<code>iadd</code>	<code>add</code>
<code>MUL</code>	<code>imul</code>	<code>mul</code>
<code>STORE</code>	<code>istore_3</code>	<code>stloc.3</code>

MiniJava =

```
"void" "main" "(" ")" "{"  
    [ VarDecl ]  
    StatSeq  
"}" .
```

VarDecl =

```
"int"  
ident↑identStr sem symbolTable.declVar(identStr); endsem  
{ ","  
    ident↑identStr sem if (!symbolTable.declVar(identStr))  
                        semanticError("multiple declaration");  
                        endsem  
} ';' .
```

StatSeq = Stat { Stat } .

Stat =

```
[
  ident↑identStr      sem String destId = identStr;
                      boolean isDecl = symbolTable.isDecl(destId);
                      if (!isDecl)
                        semanticError("var. not declared");
                      endsem

  '='
  Expr↑e             sem if (isDecl)
                      symbolTable.setVal(destId, e);
                      endsem

  |
  "print" "("
    Expr↑e           sem System.out.println(e); endsem
  ")"
]
";" .
```

Expr_{↑_e} =

Wie beim
Calculator

Term_{↑_e}

```
{  '+' Term↑t      sem e = e + t; endsem  
|  '-' Term↑t      sem e = e - t; endsem  
} .
```

Term_{↑_t} =

Fact_{↑_t}

```
{  '*' Fact↑f      sem t = t * f; endsem  
|  '/' Fact↑f      sem t = t / f; endsem  
} .
```

Wie beim
Calculator

Fact_{↑**f**} =

```
| number↑numberVal sem f = numberVal; endsem
| ident↑identStr sem if (!symbolTable.isDecl(identStr))
                        semanticError("var. not declared");
                        else
                            f = symbolTable.getVal(identStr);
                        endsem
| read "(" sem System.out.print(" > ");
                        f = Input.readInt();
                        endsem
    ")"' "
| "(" Expr↑f ")" .
```

neu

MiniJava =

"void" "main" "(" ")" "{"

 [VarDecl]

 StatSeq

"}"

sem `codeGenerator.emit(OpCode.END);` endsem

.

VarDecl =

"int"

ident_↑identStr

sem `symbolTable.declVar(identStr);` endsem

{ " , "

 ident_↑identStr

sem if (`!symbolTable.declVar(identStr)`)
 `semanticError("multiple declaration");`
endsem

} ' ; ' .

StatSeq = Stat { Stat } .

Stat =

[

```
    ident↑identStr    sem String destId = identStr;  
                      boolean isDecl = symbolTable.isDecl(destId);  
                      if (!isDecl)  
                          semanticError("variable not declared");  
                      else  
                          codeGenerator.emit(Opcode.LOAD_ADDR,  
                                              symbolTable.addrOf(destId));
```

endsem

'='

Expr

```
sem if (isDecl)  
    codeGenerator.emit(Opcode.STORE);
```

endsem

|

"print" "("

Expr

")"

```
sem codeGenerator.emit(Opcode.PRINT); endsem
```

] ";" .

Wie bei Transformat.
Infix→Postfix

Expr =

Term

```
{ '+' Term      sem codeGenerator.emit(OpCode.ADD); endsem
| '-' Term      sem codeGenerator.emit(OpCode.SUB); endsem
} .
```

Term =

Fact

```
{ '*' Fact      sem codeGenerator.emit(OpCode.MUL); endsem
| '/' Fact      sem codeGenerator.emit(OpCode.DIV); endsem
} .
```

Wie bei Transformat.
Infix → Postfix

Fact =

```
| number ↑ numberVal sem
                                codeGenerator.emit(OpCode.LOAD_CONST,
                                numberVal);
                                endsem
| ident ↑ identStr sem
                                if (!symbolTable.isDecl(identStr))
                                    semanticError("variable not declared");
                                else
                                    codeGenerator.emit(OpCode.LOAD_VAL,
                                    symbolTable.addrOf(identStr));
                                endsem
| read "(" sem
                                codeGenerator.emit(OpCode.READ);
                                endsem
    ")"' "
| "(" Expr ")" .
```

neu